



Introduction to Kubernetes (K8s) and its Concepts

Introduction

Kubernetes (K8s) is an open-source **container orchestration platform** that helps in **automating deployment, scaling, and management** of containerized applications.

- **What is Kubernetes?**
- **Why do we need Kubernetes?**
- **What are the key components of Kubernetes?**

This guide will help you **understand Kubernetes concepts and how it simplifies container management**.

Section 1: Learn

What is Kubernetes?

Kubernetes is a **container orchestration system** that **manages multiple containers** across clusters of machines. It automates:

- **Deployment of applications**
- **Scaling of workloads**
- **Networking and storage management**
- **Self-healing of applications**

Why Use Kubernetes?

Without Kubernetes	With Kubernetes
Manually manage containers	Automates container management



Without Kubernetes	With Kubernetes
No built-in load balancing	Built-in service discovery & load balancing
Hard to scale applications	Auto-scaling support
Risk of application failures	Self-healing of applications

Key Features of Kubernetes

- **Automated Deployment** – Easily deploy applications using YAML files.
- **Load Balancing** – Distributes traffic to maintain high availability.
- **Self-Healing** – Restarts failed containers automatically.
- **Auto-Scaling** – Adjusts resources based on demand.
- **Rolling Updates** – Deploy updates without downtime.

Interesting Fact

Kubernetes was originally developed by **Google**, based on its internal **Borg system**, and later donated to the **Cloud Native Computing Foundation (CNCF)**.

Section 2: Practice

1. Kubernetes Architecture

Kubernetes follows a **Master-Worker Node Architecture**:



Component	Description
Master Node	Manages the cluster and schedules workloads
Worker Node	Runs application containers
Pods	Smallest unit that runs a container
Services	Exposes applications to the network
Deployments	Manages updates and scaling of applications

2. Setting Up Kubernetes (Minikube)

For local testing, we can use **Minikube** to set up Kubernetes on a single machine.

Step 1: Install Minikube

For Ubuntu/Linux:

```
curl -LO  
https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64  
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Step 2: Start Minikube Cluster

```
minikube start
```



Step 3: Verify Cluster Status

```
kubectl cluster-info
```

```
kubectl get nodes
```

3. Deploying an Application in Kubernetes

Step 1: Create a Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
```



```
- containerPort: 80
```

Apply the deployment:

```
kubectl apply -f deployment.yaml
```

Step 2: Expose the Deployment

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80
```

Step 3: Check Running Pods

```
kubectl get pods
```

Step 4: Access the Application

Find the service port:

```
kubectl get services
```

Access via **http://localhost:**

4. Scaling Applications in Kubernetes

To scale up the replicas:

```
kubectl scale deployment nginx-deployment --replicas=5
```

To check the updated replicas:

```
kubectl get deployments
```



Section 3: Know More

Frequently Asked Questions

1. **What is the difference between Docker and Kubernetes?**

- **Docker** is a container runtime, while **Kubernetes** is a container orchestrator that manages Docker containers.

2. **What is a Pod in Kubernetes?**

- A **Pod** is the smallest deployable unit in Kubernetes that runs one or more containers.

3. **How do I delete a Kubernetes deployment?**

```
kubectl delete deployment nginx-deployment
```

4. **What is a Kubernetes Service?**

- A **Service** exposes a set of Pods to the network, allowing external access.

5. **Can Kubernetes run without Docker?**

- Yes, Kubernetes supports **other container runtimes** like **containerd** and **CRI-O**.

Conclusion

Kubernetes simplifies **container orchestration** by managing **deployment, scaling, and networking**.

Start by **setting up Kubernetes, deploying applications, and scaling workloads**, and soon you'll be **orchestrating cloud-native applications efficiently!**